

Руководство по жизненному циклу проекта через LLM

Как подготовить окружение и управлять LCD-bitmap IDE через Claude Code, Codex, OpenCode, LM Studio и Ollama.

- | | | |
|-------------------------------------|-----------------------------|--------------------------------------|
| 1. Что должен делать LLM | 2. Подготовка окружения | 3. Выбор connector-пути |
| 4. Рекомендуемый system prompt | 5. Чтение состояния проекта | 6. Безопасное редактирование проекта |
| 7. Валидация измерительных workflow | 8. Compile и review | 9. Release handoff |
| 10. Security rules | | |

1. Что должен делать LLM

Используйте LLM как помощника для структурированных правок, review и повторяемой работы с проектом. UI остаётся источником истины.

LLM сначала читает состояние проекта, предлагает небольшое изменение, применяет его через MCP или REST, затем валидирует и сообщает результат.

- ☐ Сначала читать, потом писать.
- ☐ Менять один концепт за раз.
- ☐ Валидировать после каждой пачки.
- ☐ Держать desktop app открытым.

2. Подготовка окружения

- 1. Установите Node.js 22+, Git и выбранный LLM-клиент.
- 2. Выполните `npm ci` в репозитории.
- 3. Запустите Electron app через `npm run electron:dev`.
- 4. Откройте проект или `bundled demo` до вызова `tools`.

```
npm ci
npm run electron:dev
curl http://127.0.0.1:8766/api/health
```

3. Выбор connector-пути

Клиент	Лучший путь	Fallback
Claude Code	MCP HTTP server	REST через shell-команды
Codex	MCP HTTP server или shell REST calls	Небольшие JSON scripts
OpenCode	MCP HTTP server	curl commands
LM Studio	Tool wrapper поверх REST	Ручные curl recipes
Ollama	Agent wrapper поверх REST	Scripted batch edits

4. Рекомендуемый system prompt

Дайте agent явные ограничения. Он не должен удалять states, перезаписывать проект или делать широкие правки без подтверждения.

```
You are operating LCD-bitmap IDE through local MCP/REST.  
First read the project summary and FSM states.  
Before each mutation, describe the intended change.  
Apply small changes only.  
After each mutation, validate and summarize the result.  
Do not delete states unless the user explicitly asks.
```

5. Чтение состояния проекта

Каждую сессию начинайте с чтения проекта. Это предотвращает устаревшие предположения и правки не того state, screen или tag.

```
curl http://127.0.0.1:8766/api/project/meta  
curl http://127.0.0.1:8766/api/fsm/states  
curl http://127.0.0.1:8766/api/tags
```

6. Безопасное редактирование проекта

1. Запрашивайте один outcome: добавить state, переименовать screen, добавить tag или связать transition.
2. Пусть agent подготовит payload.
3. Примените payload через MCP или REST.
4. Снова прочитайте state и сравните результат.
5. После успешной пачки сохраните проект в UI.

7. Валидация измерительных workflow

Для measurement-oriented проектов LLM должен проверить наличие setup, run, result, cancel и error paths. Также нужно проверить, что measurement tags имеют units и precision.

- ❑ Initial state существует.
- ❑ START ведёт по валидному пути.
- ❑ Result state достижим.
- ❑ Cancel и error paths достижимы.
- ❑ Measurement tags имеют names, units и precision.

8. Compile и review

1. Компилируйте screens после LCD-изменений.
2. Запускайте Runtime Preview после FSM-изменений.
3. Проверяйте screenshots или exported assets до передачи firmware-разработчику.
4. Коммитьте docs и project changes вместе, если они описывают одно поведение.

9. Release handoff

Чистая передача включает .lcdproj, exported firmware assets, screenshots, validation notes и точную версию приложения или commit hash, которыми они были созданы.

10. Security rules

- ❑ Не открывайте 127.0.0.1 automation ports через публичный tunnel.
- ❑ Не вставляйте private implementation files, device identifiers или keys в prompts.
- ❑ Проверяйте каждый DELETE вручную.
- ❑ Держите proprietary projects вне public branch.

Generated from scripts/generate-user-docs.mjs. For automation details see docs/API_MCP_CONNECTORS.md.